

## • O papel do compilador e as suas limitações

- O compilador é a ferramenta que traduz a linguagem para a linguagem da máquina. Ele tenta otimizar o código automaticamente, mas é conservador, ou seja, se houver o mínimo risco de a otimização mudar o que o programa faz, ele não faz.
- A otimização é importante para melhorar a velocidade, consumo de energia e no tamanho do código.
- Existem dois grandes "bloqueadores" que impedem o compilador de ajudar:

+ **Aliasing de memória:** Quando dois ponteiros podem estar a apontar para o mesmo sítio na memória. O compilador não arrisca simplificar as células porque não sabe se alterar um vai afetar o outro

Exemplo: Se os ponteiros  $*x$  e  $*y$  forem iguais, uma operação como  $*x += *y$  ao invés de quadruplicar o valor enquanto numa versão otimizada poderia apenas triplicar gerando erros de lógica.

+ **Efeitos secundários:** Se uma função altera uma variável global (fora dela), o compilador não pode saltar parâmetros ou reutilizar resultados, pois isso mudaria o estado do programa

## • Otimizações Independentes da Máquina

### + Movimentação de Código (Code Motion)

Se tens um cálculo dentro de um ciclo (loop) que dá sempre o mesmo resultado em todas as voltas, move-o para fora do ciclo.

Exemplo:

- Antes: Calcular  $x = y + z$  1000 vezes dentro do loop
- Depois: Calcular uma vez antes do loop começar e usar o resultado dentro

### + Redução de Acessos à Memória

Ler e escrever na memória RAM é muito mais lento do que usar registadores

Exemplo:

Em vez de atualizar uma variável na memória em cada volta de um loop, usa uma variável local e só guarda o valor final na memória quando o loop acabar

### + Desenrolar de Ciclos

Gerar um ciclo gasta muito tempo então o "loop unrolling" consiste em fazer mais trabalho em cada volta para reduzir o número total de voltas.

Vantagem → fica mais rápido

Desvantagem → ocupa mais memória

### + Inlining

Chamar uma função tem um "custo" de tempo (overhead). O inlining substitui a chamada da função pelo próprio código da função diretamente no local. É ideal para funções mais pequenas

## • Otimizações dependentes da Máquina

Estas instruções dependem do processador específico.  
Em IA32:

- Substituição de instruções → Por vezes, há duas formas de fazer o mesmo. Por exemplo, colocar um registador a zero pode ser feito com `mov ax, 0x0`. O `xor` continua a ser preferido pois ocupa menos espaço.

- Matemática "inteligente" → Multiplicar é algo muito lento por isso se multiplicarmos por uma potência de base 2, o compilador substitui isso por um "shift" de bits que é instantâneo. Em números que não são potências de base 2, o compilador decompõe o número em potências de base 2.

## • Otimização para a cache (Localidade)

A cache guarda cópias de informação temporárias para além do acesso pela RAM, e para este acesso ser rápido, a cache tem de ter uma boa localidade

1. Localidade temporal: Acesso aos dados usados recentemente

2. Localidade espacial: Acesso a dados que estão vizinhos na memória

Exemplo:

As matrizes são guardadas em linha. Se percorrermos linha a linha, o processador já terá os próximos dados na cache enquanto que se percorrermos coluna a coluna temos de "saltar" na memória e o programa será mais lento

Quase todas as otimizações requerem uma escolha:

⊕ Velocidade ⇒ ⊕ tamanho ou ⊖ legibilidade do código